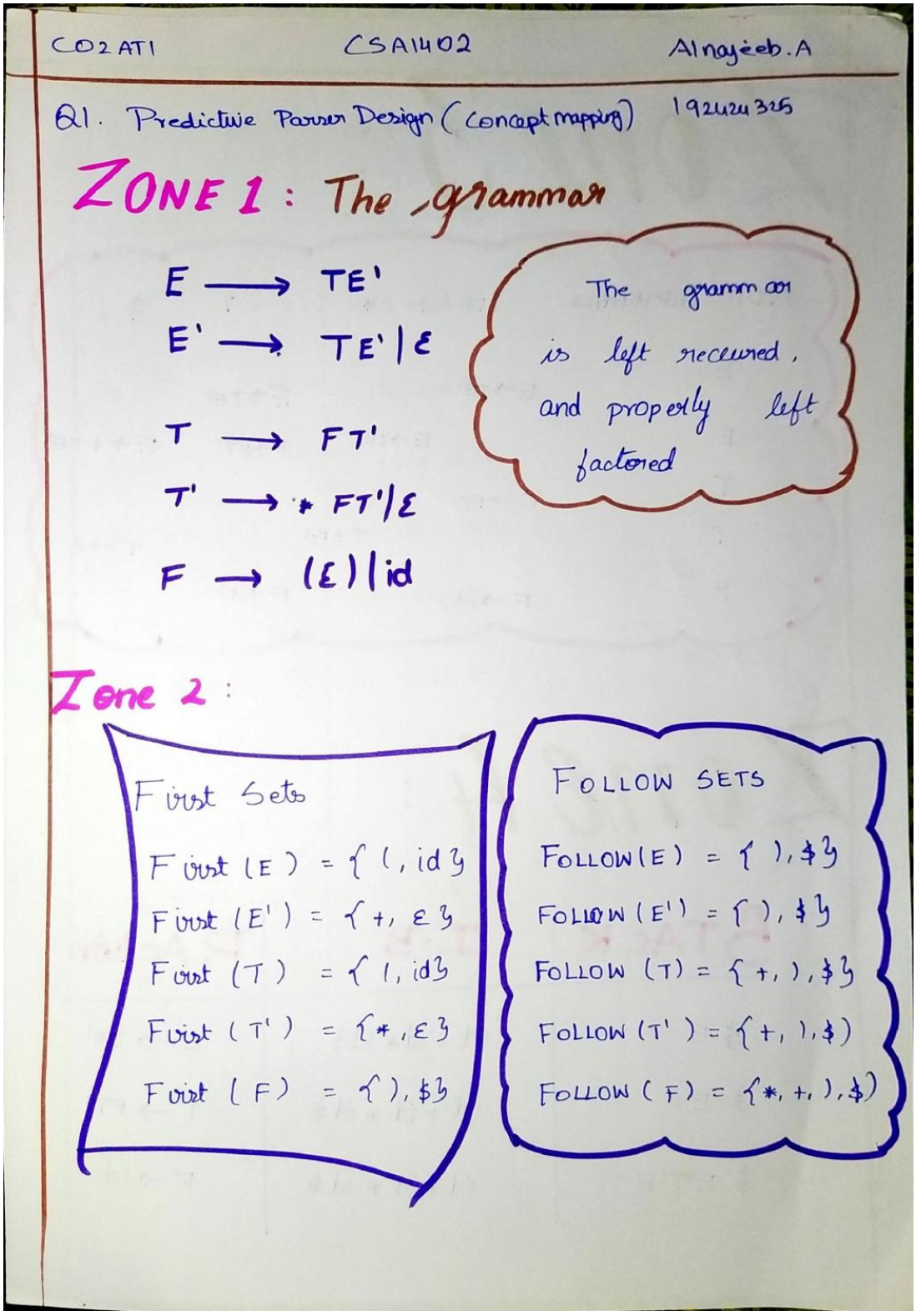
ASSESSMENT TOOL 1

GIT HUB LINK:https://github.com/Najeeb2006/compiler-design/tree/main/CO2_AT1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design**

AL NAJEEB A
192424325



Zone 3 :

Non Terminals	ID	+	*	(	)	$
E				$E \to TE'$		
E'			$E \to TE'$		$E \to +TE'$	
T				$T \to FT'$		
T'			$T \to FT'$			$T' \to \varepsilon$
F	$F \to id$			$F \to (t)$		

Zone 4 :

STACK	I.B	P. Action
$ E	id + id * id $	$E \to TE'$
$ E'T	id + id * id $	$T \to FT'$
$ E'T'F	id + id * id $	$F \to id$

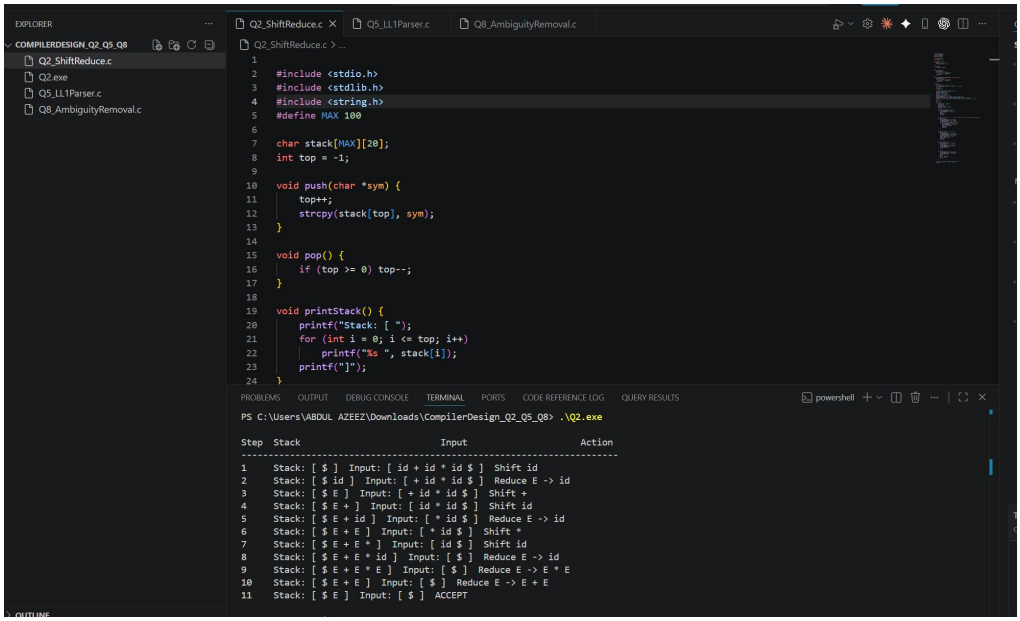
$ E'T' id	id + id * id $	match id
$ E'T'	+ id * id $	T' → ε
$ E'	+ id * id $	E' → TE'
$ E'T +	id + id $	Match +
$ E'T'F	id * id $	T → FT'
$ E'T' id	id + id $	match id
$ E'T'	* id $	T' → * FT'
$ E'T'F*	* id $	match *
$ E'T'F	id $	F → id
$ E'T' id	id $	match id
$ E'T'	$	T' → ε
$ E'	$	E' → ε

Snippet OF

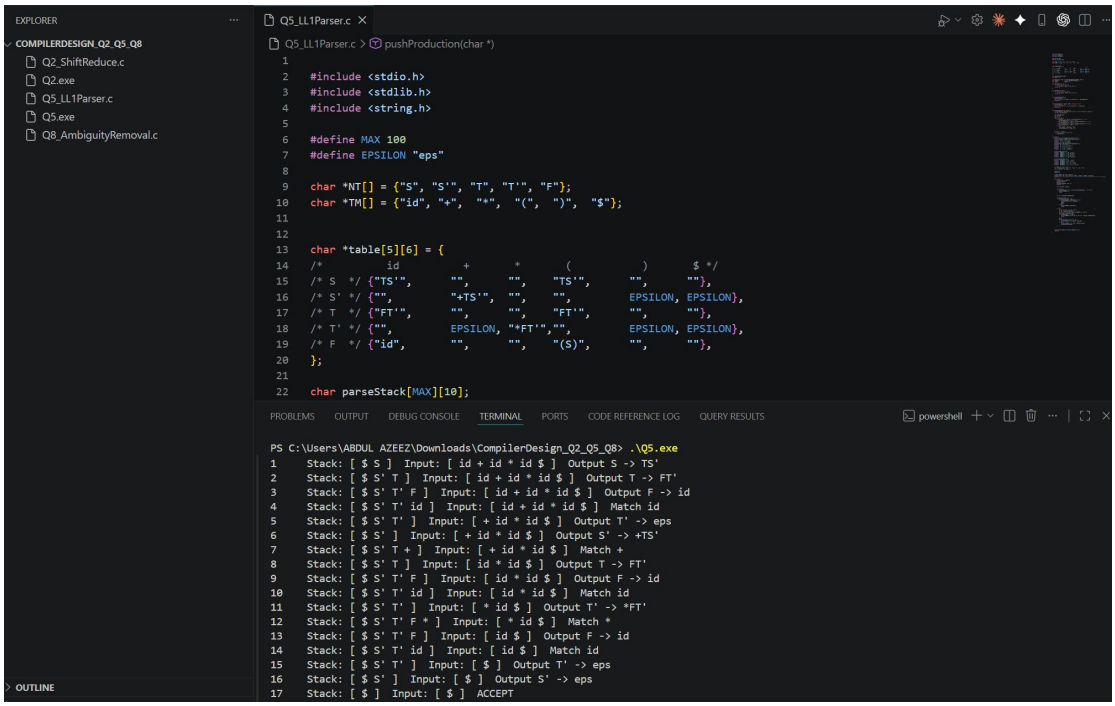
PREDICTIV PARSER

```
Class  Predictive parser :
    def __init__ ( self , start ) :
        self . elements = [$ , start ]
    def  push ( self , production ):
        Self . element . extend ( reversed ( prod ))
    def  match (self , current_token ):
        if  self - elements [-1] == current_tok ;
            return self . element . pop()
```

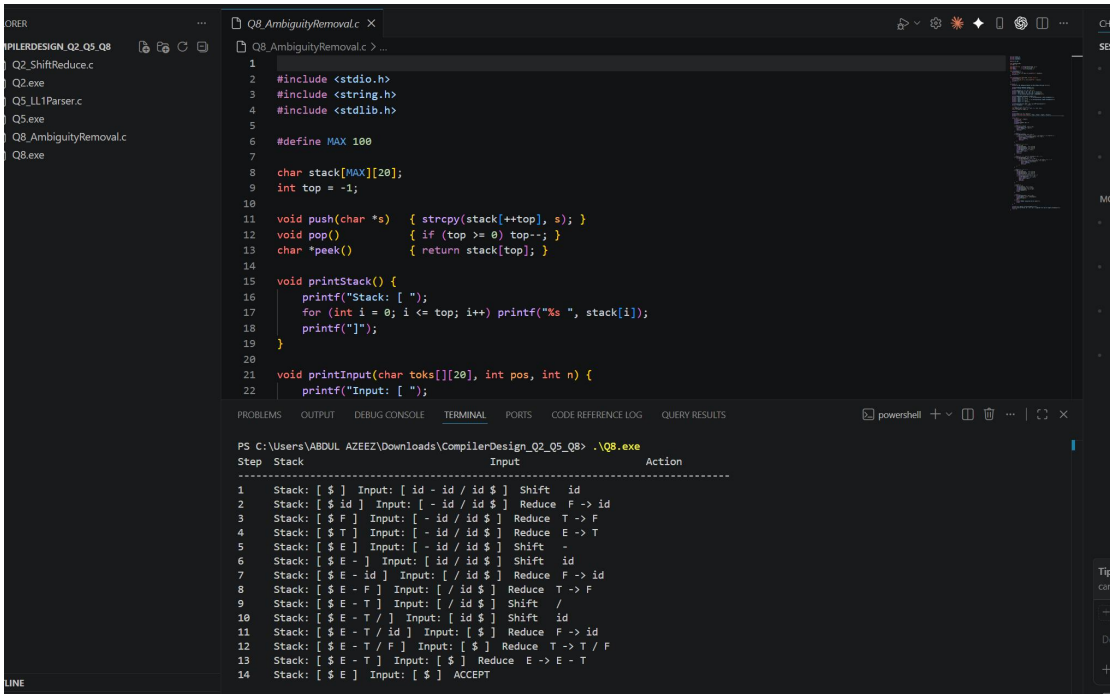
Q2. Shift-Reduce Parsing (Application Level)



Q 5. Advanced LL(1) Parser Evaluation and Design



Q8. Ambiguity Removal and Parsing Strategy Evaluation


Q8_AmbiguityRemoval.c
Q2_ShiftReduce.c
Q2.exe
Q5_LL1Parser.c
Q5.exe
Q8_AmbiguityRemoval.c
Q8.exe
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 100
char stack[MAX][20];
int top = -1;
void push(char *s) { strcpy(stack[++top], s); }
void pop() { if (top >= 0) top--; }
char *peek() { return stack[top]; }
void printStack() {
printf("Stack: [");
for (int i = 0; i <= top; i++) printf("%s ", stack[i]);
printf("]");
}
void printInput(char toks[][20], int pos, int n) {
printf("Input: [");
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG QUERY RESULTS
powershell
PS C:\Users\ABDUL AZEEZ\Downloads\CompilerDesign_Q2_Q5_Q8> .\Q8.exe
Step Stack Input Action
1 Stack: [$] Input: [id - id / id $] Shift id
2 Stack: [$ id] Input: [- id / id $] Reduce F -> id
3 Stack: [$ F] Input: [- id / id $] Reduce T -> F
4 Stack: [$ T] Input: [- id / id $] Reduce E -> T
5 Stack: [$ E] Input: [- id / id $] Shift -
6 Stack: [$ E -] Input: [id / id $] Shift id
7 Stack: [$ E - id] Input: [/ id $] Reduce F -> id
8 Stack: [$ E - F] Input: [/ id $] Reduce T -> F
9 Stack: [$ E - T] Input: [/ id $] Shift /
10 Stack: [$ E - T /] Input: [id $] Shift id
11 Stack: [$ E - T / id] Input: [$] Reduce F -> id
12 Stack: [$ E - T / F] Input: [$] Reduce T -> T / F
13 Stack: [$ E - T] Input: [$] Reduce E -> E - T
14 Stack: [$ E] Input: [$] ACCEPT